



ΕΘΝΙΚΟ ΜΕΤΣΟΒΙΟ ΠΟΛΥΤΕΧΝΕΙΟ, ΣΧΟΛΗ ΣΗΜΜΥ
ΒΑΣΕΙΣ ΔΕΔΟΜΕΝΩΝ, ΕΞΑΜΗΝΙΑΙΑ ΕΡΓΑΣΙΑ
ΑΚΑΔΗΜΑΪΚΟ ΕΤΟΣ 2025-26

Ομάδα Project 126:

1. Τσακαλίδου Μαρία, 09122438, ΣΕΜΦΕ
2. Χατζηιωάννου Μαρία Αυρηλία, 0912241, ΣΕΜΦΕ

Ημερομηνία παράδοσης: 17/05/2026

ασθενής μπορεί αν έχει πολλαπλές νοσηλείες, αλλά αυτές θα συνδέονται και με πολλαπλές διαδικασίες διαλογής. Επομένως η πληροφορία στη σχέση hospitalization.triage_bed μαζί με της σχέση patient.triage_nurse αρκεί.

- Στην διατύπωση της εκφώνησης αναφέρεται ότι η φαρμακευτική αγωγή χορηγείται απο έναν γιατρό και λαμβάνεται απο έναν ασθενή. Αυτό υποδηλώνει μια σχέση μεταξύ αυτών των τριών οντοτήτων, όμως επειδή μια συνταγογράφηση μπορεί να δοθεί σε ασθενή μόνο κατά τη διάρκεια νοσηλείας, η πληροφορία αυτή ενσωματώθηκε στη σχέση hospitalization.prescription_doctor.
- Δημιουργήθηκαν δύο οντότητες για τα δύο διαφορετικά είδη αξιολόγησης που μπορεί να κάνει κάθε ασθενής με ολοκληρωμένη νοσηλεία: evaluation_of_hospitalization και evaluation_of_doctor.

1.2 Σχεσιακό διάγραμμα, ανάπτυξη ΒΔ

Με βάση τη μετατροπή του παραπάνω ER μοντέλου σε σχεσιακό, προκύπτει το παρακάτω σχεσιακό διάγραμμα της βάσης δεδομένων:

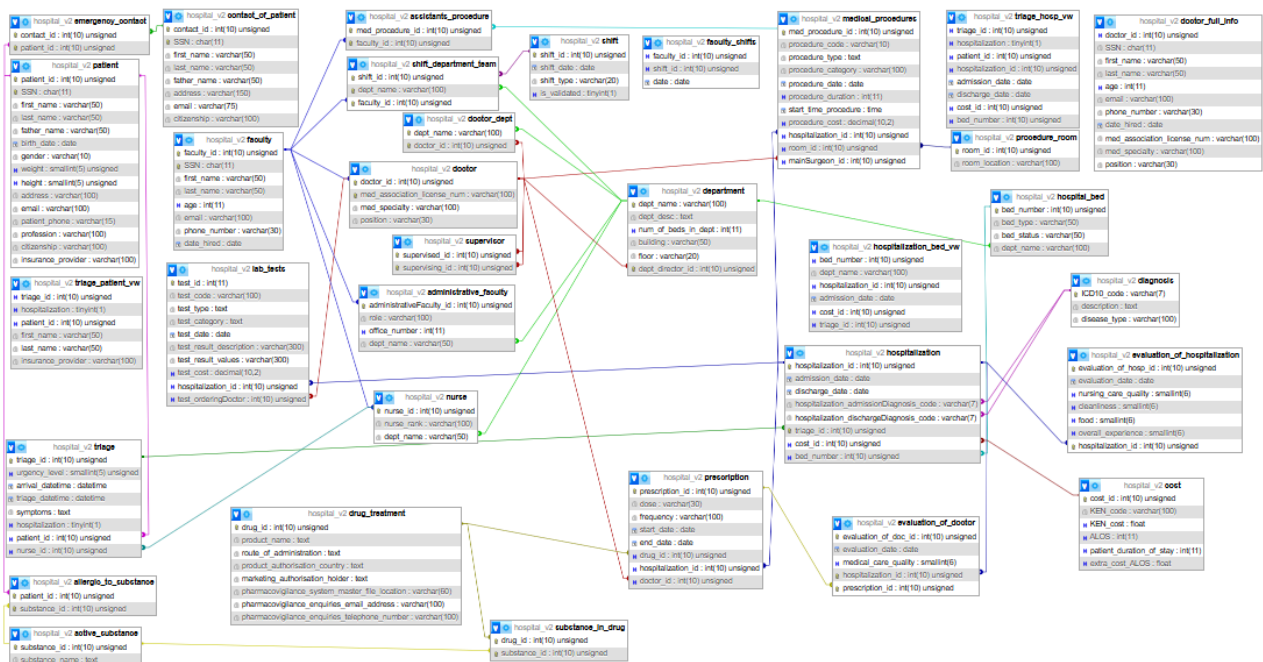


Figure 1: Το σχεσιακό διάγραμμα της βάσης δεδομένων

Η εξαγωγή του παραπάνω διαγράμματος έγινε μέσω της εφαρμογής XAMPP (συγκεκριμένα από το περιβάλλον του phpMyAdmin).

- Σχέσεις ένα-προς-πολλά (one-to-many) που μετατράπηκαν σε ξένα κλειδιά (foreign keys): Στον πίνακα nurse προστέθηκε το πρωτεύον κλειδί του department (nurse.dept), στον πίνακα administrative_faculty προστέθηκε το πρωτεύον κλειδί του department (administrativeStaff_dept), στον πίνακα triage προστέθηκαν ως ξένα κλειδιά τα πρωτεύοντα κλειδιά των nurse και patient (patient.triage_nurse), στον πίνακα hospitalization προστέθηκε το πρωτεύον κλειδί του triage (hospitalization.triage), στον πίνακα medical_procedures

προστέθηκε το `hospitalization_id` ως ξένο κλειδί (`hospitalization_medProcedures`), στον πίνακα `hospitalization` προστέθηκε το αντίστοιχο αναγνωριστικό διάγνωσης εισαγωγής (`hospitalization_admissionDiagnosis_code`) καθώς και το αναγνωριστικό διάγνωσης εξιτηρίου (`hospitalization_dischargeDiagnosis_code`), στον πίνακα `lab_tests` προστέθηκε το `doctor_id` ως ξένο κλειδί (`test_OrderingDoctor`), στον πίνακα `medical_procedure` προστέθηκε το `doctor_id` ως ξένο κλειδί (`mainSurgeon_procedure`), στον πίνακα `evaluation_of_doctor` προστέθηκε το `prescription_id` ως ξένο κλειδί (`prescription_evaluation_of_doc`), και στον πίνακα `prescription` προστέθηκε το αναγνωριστικό του φαρμάκου (`drug_prescription`).

- Σχέσεις ένα-προς-ένα (one-to-one) που μετατράπηκαν σε ξένα κλειδιά: Στον πίνακα `department` προστέθηκε το αντίστοιχο `doctor_id` ως ξένο κλειδί (`dept_director`), στον πίνακα `hospital_bed` προστέθηκε το αναγνωριστικό του τμήματος (`bed_department`), στον πίνακα `hospitalization` προστέθηκε το `cost_id` (`hospitalization_cost`) καθώς και το αναγνωριστικό της εξέτασης (`hospitalization_test`), στον πίνακα `medical_procedures` προστέθηκε το `room_id` (`procedure_in_room`), στον πίνακα `evaluation_of_hospitalization` προστέθηκε το `hospitalization_id` ως ξένο κλειδί (`hospitalization_evaluation`), στον πίνακα `medical_procedure` προστέθηκε το `room_id` ως ξένο κλειδί (`procedure_in_room`), και στον πίνακα `evaluation_of_doctor` προστέθηκε το `doctor_id` (`prescription_evaluation`).
- Σχέσεις πολλά-προς-πολλά (many-to-many) που υλοποιήθηκαν μέσω νέων σχημάτων, με γνωρίσματα τα πρωτεύοντα κλειδιά των οντοτήτων που συμμετέχουν στην αντίστοιχη σχέση: `supervisor`, `doctor_dept`, `emergency_contact`, `assistant_procedure`, `allergic_to_substance`, `substance_in_drug`, `shift_department_team`.
- Aggregated σχέση των `hospitalization-diagnosis`: προστέθηκαν στο table του entity `hospitalization` δύο attributes για τους κωδικούς των δύο διαγνώσεων (`admission` και `discharge`, καθεμία είναι μοναδική για δεδομένο `hospitalization`). Οι υπόλοιπες σχέσεις μετατράπηκαν όπως περιγράφεται παραπάνω.
- Τριαδικές σχέσεις: Οι σχέσεις `patient_triage_nurse`, `hospitalization_prescription_doctor` και `hospitalization_triage_bed` που δεν είναι πληθικότητας "many" σε κάθε μεριά (1. σε κάθε ασθενή αντιστοιχεί ένας νοσοκόμος για κάθε διαλογή 2. σε κάθε νοσηλεία αντιστοιχεί ένας γιατρός για κάθε συνταγογράφηση 3. κάθε νοσηλεία αντιστοιχεί σε μια συγκεκριμένη διαλογή και σχετίζεται με συγκεκριμένη κλίνη, άρα έχουμε arrows προς τις οντότητες 1. nurse 2. doctor 3. hospital_bed αντίστοιχα) μετατράπηκαν σε attributes (1. `patient_id`, `nurse_id` στο `triage`, 2. `hospitalization_id`, `doctor_id` στο `prescription`, `triage_id`, 3. `bed_number` στο `hospitalization`). Η σχέση `shift_department_team` που είναι πληθικότητας "many" σε κάθε μεριά μετατράπηκαν σε πίνακες με κλειδιά τα id και των τριών σχέσεων κάθε φορά.

1.2.1 Απλοί περιορισμοί

Οι περιορισμοί αναλύονται παρακάτω άνα πίνακα στον οποίο ανήκουν.

1. `faculty`: `ssn` UNIQUE για να εξασφαλίσουμε ότι το AMKA του κάθε ατόμου είναι μοναδικό, `email_constraint` για να εξασφαλίσουμε ότι το email έχει την σωστή μορφή.
2. `doctor:med_association_license_num` UNIQUE για να εξασφαλίσουμε ότι κάθε ιατρός έχει μοναδικό αριθμό άδειας ιατρικού συλλόγου, `correct_rank_doc` για έλεγχο έγγυρης βαθμίδας, `doctor_fk`

3. nurse:nurse_fk,nurse_dept_fk
4. administrative_faculty:adm_fk,adm_dept_fk
5. hospital_bed:bed_dept_fk
6. department:-
7. shift:-
8. triage:fk_triage_patient,fk_triage_faculty
9. contact_of_patient:ssn UNIQUE εξασφάλιση μοναδικότητας ,email_contact_patient για να εξασφαλίσουμε ότι το email έχει την σωστή μορφή.
10. supervisor:supervised_fk,supervisor_fk
11. doctor_dept:facultyid_fk,dept_fk
12. shift_department_team:shifftid_fk,faculty_id_fk,dept_fk
13. emergency_contact:fk_emergency_con_contact_of_patient ,T fk_emergency_contact_patient
14. patient: constraint_for_ssn εξασφαλίζει τον κανόνα το length του ssn να είναι 11, email_patient εξασφαλίζει τη σωστή μορφή του email.
15. hospitalization: έχει τον περιορισμό discharge_date_constraint που εξασφαλίζει οτι η ημερομηνία admission είναι πριν την ημερομηνία discharge, και τα παρακάτω foreign keys που προκύπτουν απο το σχήμα: fk_hospitalization_cost, fk_hospitalization_bed, fk_hospitalization_triage, fk_hospitalization_admissionDiagnosis_code, fk_hospitalization_dischargeDiagnosis_code.
16. diagnosis: -
17. lab_tests: έχει τα παρακάτω foreign keys που προκύπτουν απο το σχήμα: fk_lab_tests_hospitalization, fk_lab_tests_test_orderingDoctor.
18. medical_procedures: έχει τα παρακάτω foreign keys που προκύπτουν απο το σχήμα: fk_hospitalization_medProcedures, fk_medical_procedures_room, fk_mainSurgeon_procedure.
19. procedure_room: -
20. active_substance: -
21. drug_treatment: email_drug_treatment εξασφαλίζει οτι τα email έχουν σωστή μορφή.
22. prescription: fk_prescription_patient, fk_prescription_doctor.
23. evaluation_of_hospitalization: fk_evaluation_hospitalization.
24. cost: fk_evaluation_doc_hospitalization, fk_evaluation_doctor
25. evaluation_of_doctor: έχει τα παρακάτω foreign keys που προκύπτουν απο το σχήμα: fk_evaluation_doc_hospitalization, fk_evaluation_doctor.
26. assistants_procedure: fk_med_procedure_assistants_procedure, fk_faculty_assistants_procedure.

- 27. allergic_to_substance: fk_active_substance_allergic_to_substance, fk_patient_allergic_to_substance.
- 28. substance_in_drug: fk_drug_treatment_substance_in_drug, fk_active_substance_substance_in_drug.

1.2.2 Triggers και Συναρτήσεις

Γράφτηκαν κάποια απλά triggers με σκοπό να εξασφαλίσουν την ακεραιότητα του πεδίου τιμών κάποιων στηλών, π.χ. της στήλης rank στο nurse: 'validate_nurse_rank', 'valid_shift_type', 'valid_urgency_level'.

Επίσης γράφτηκαν trigger που απαγορεύουν τη διαγραφή σε πίνακες που θεωρήσαμε ότι το νοσοκομείο δεν επιτρέπει να χανθεί αυτή η πληροφορία από το σύστημα. Συγκεκριμένα, στους πίνακες: department, faculty, doctor, nurse, administrative_faculty, shift, triage, contact_of_patient, patient, hospitalization, diagnosis, lab_tests, medical_procedure, procedure_room, active_substance, drug_treatment, prescription, cost, shift_department_team, emergency_contact, assistants_procedure, substance_in_drug.

Επίσης γράφτηκαν τα παρακάτω triggers, που εξασφαλίζουν πιο σύνθετους business rules:

1. faculty:-
2. doctor:-
3. nurse:-
4. administrative_faculty:-
5. hospital_bed:
'hospitalization_check_hospital_beds' before insert on hospitalization, έλεγχος αν μια κλίνη είναι διαθέσιμη και αν είναι κάνει την αντίστοιχη ενημέρωση σε μη διαθέσιμη.
6. department:
 - i. 'directing_only_one_dept' before insert on department, εξασφαλίζει ότι δεν υπάρχει γιατρός που να διευθύνει πάνω από 1 τμήμα τη φορά.
 - ii. 'validate_director_rank' before insert on department, ελέγχει εάν ο γιατρός που καταχωρούμε ως διευθυντή έχει το σωστό βαθμό ως clinical director.
7. shift:
 - i. 'validation_of_shift' before update on shift, εξασφαλίζει ότι μια ομάδα εφημερίας τμήματος καλύπτεται από τουλάχιστον 3 ιατρούς, 6 νοσηλευτές και 2 διοικητικούς υπαλλήλους.
 - ii. 'intern_doc_should_not_work_alone' before update on shift, εξασφαλίζει ότι σε βάρδια που συμμετέχει ειδικευόμενος ιατρός υπάρχει τουλάχιστον ένας ιατρός βαθμίδας Επιμελητή Α' ή Διευθυντής
8. triage:-
9. contact_of_patient:-

10. supervisor:
 - i. 'supervisor_checks' before insert on supervisor, εξασφαλίζει ότι ένας ιατρός εποπτεύει μέχρι 5 άτομα, ένας διευθυντής δεν έχει επόπτη, ένας ειδικευόμενος δεν μπορεί να είναι επόπτης, ένας επιμητής B δεν μπορεί να επιβλέπει επιμελητές A, αποφυγή κυκλικής αλυσίδας εποπτείας και κάποιος δεν μπορεί να είναι επόπτης του εαυτού του.
11. doctor_dept:-
12. shift_department_team:
 - i. 'max_shift_for_faculty' before insert on shift_department_team, εξασφαλίζει ότι τηρούνται τα μέγιστα όρια βαρδιών ανά μήνα ανά κατηγορία προσωπικού, με βάση τα όρια που θέτει η εκφώνηση της εργασίας.
 - ii. 'check_shift_rest_period' before insert on shift_department_team, εξασφαλίζει το ελάχιστο διάστημα ανάπαυσης 8 ωρών μεταξύ δυο διαδοχικών βαρδιών του ίδιου μέλους προσωπικού.
 - iii. 'check_staff_placement' before insert on shift_department_team, εξασφαλίζει ότι ένας εργαζόμενος δεν εργάζεται σε μια βάρδια σε δύο τμήματα και ότι τα μέλη νοσηλευτών και διοικητικού προσωπικού εργάζονται αποκλιστηρά στο τμήμα που ανήκουν.
 - iv. 'check_max_3_night_shifts' before insert on shift_department_team, εξασφαλίζει ότι κανένα μέλος προσωπικού δεν μπορεί να συμμετέχει σε περισσότερες από 3 συνεχόμενες νυχτερινές βάρδιες.
13. emergency_contact:-
14. patient: -
15. hospitalization:
 - i. trigger, after update on hospitalization: 'upd_cost_after_discharge', το οποίο, μετά από αλλαγή του discharge date της νοσηλείας, ανανεώνει στον πίνακα cost την τιμή του επιπλέον κόστους νοσηλείας, σε περίπτωση που ο ασθενής έχει υπερβεί το ΜΔΝ.
 - ii. trigger, before insert on hospitalization: 'unique_hospitalization', το οποίο ελέγχει εάν ο ασθενής νοσηλεύεται ήδη στο νοσοκομείο.
16. diagnosis: -
17. lab_tests: -
18. medical_procedures:
 - i. 'valid_date_procedure', before insert on medical_procedures, εξασφαλίζει ότι η ημερομηνία της επέμβασης είναι κατά τη διάρκεια νοσηλείας του ασθενή.
 - ii. 'valid_procedure_room', before insert on medical_procedures, εξασφαλίζει ότι το χειρουργείο δεν είναι κατειλημμένο την ώρα της επέμβασης.
 - iii. 'valid_surgeon_shift' before insert on 'medical_procedures', εξασφαλίζει ότι ο χειρουργός έχει βάρδια την ώρα της επέμβασης.
 - iv. 'main_surgeon_no_double_booking', before insert on medical_procedures, εξασφαλίζει ότι ο χειρουργός δεν εκτελεί άλλη επέμβαση την ίδια ώρα με την νέα επέμβαση.

19. procedure_room: -
20. active_substance: -
21. drug_treatment: -
22. prescription: i. trigger, before insert on prescription: 'valid_prescription_no_allergy', εξασφαλίζει ότι ο ασθενής δεν έχει αλλεργία σε κάποια δραστική ουσία που περιέχεται στο συνταγογραφούμενο φάρμακο.
ii. trigger, before insert on prescription: 'valid_date_prescription', το οποίο ελέγχει εάν η ημερομηνία έναρξης της φαρμακευτικής αγωγής του ασθενούς είναι μεταξύ admission και discharge date της νοσηλείας.
23. evaluation_of_hospitalization: i. 'valid_date_evaluation1', before insert, εξασφαλίζει ότι η νοσηλεία που αξιολογείται έχει ολοκληρωθεί όταν γίνεται η αξιολόγηση της απο τον ασθενή.
24. cost: -
25. evaluation_of_doctor: i. 'valid_date_evaluation2', before insert, εξασφαλίζει ότι η νοσηλεία κατά την οποία έγινε η συνταγογράφηση (αλληλεπίδραση με γιατρό που συνταγογραφεί) που αξιολογείται έχει ολοκληρωθεί όταν γίνεται η αξιολόγηση της απο τον ασθενή.
26. assistants_procedure: i. 'assistants_no_double_booking' before insert on assistants_procedure, εξασφαλίζει ότι κανείς απο τους βοηθούς του κυρίως χειρουργου δεν έχει κανονισμένη άλλη επέμβαση την ίδια ώρα με την νέα επέμβαση.
ii. 'valid_assistants_shift' before insert on assistants_procedure, εξασφαλίζει το προσωπικό που βοηθά στο χειρουργείο έχει βάρδια την ώρα της επέμβασης.
27. allergic_to_substance: -
28. substance_in_drug: -

Για την υλοποίηση ορισμένων trigger χρησιμοποιήσαμε και τις παρακάτω συναρτήσεις:

- FUNCTION count_supervising: Δέχεται ως είσοδο το id του supervisor και πολογίζει πόσα άτομα εποπτεύει ένας ιατρός.
- FUNCTION position_of_doctor(doc_id INT): Δέχεται το id ενός γιατρού και επιστρέφει το position του.
- FUNCTION faculty_category(id INT): Δέχεται το id ενός εργαζομένου και επιστρέφει την κατηγορία που ανήκει (γιατρός, νοσηλευτής, διοικητικό προσωπικό).

1.2.3 Views

1. view faculty_shift: ένα view που αποθηκεύει για κάθε μέλος του προσωπικού τις βάρδιες (shift_id) του και την ημερομηνία που ήταν αυτές οι βάρδιες. Δημιουργήθηκε γιατί φάνηκε χρήσιμο για τα ερωτήματα 2,8,12.
2. view doctor_full_info: ένα view που αποθηκεύει όλες τις πληροφορίες κάθε γιατρού (αντί να απαιτείται να κάνουμε explicitly join με το faculty για να τις πάρουμε). Χρησιμοποιήθηκε για το ερώτημα 4.
3. view triage_patient_vw: ένα view που συνδέει την διαλοδή με ενός ασθενή με πληροφορίες του ίδιου του ασθενή και δημιουργήθηκε για τα ερωτήματα 1,3.
4. view triage_hosp_vw : ένα view που συνδέει την διαλογή ενός ασθενή με την αντίστοιχη νοσηλεία το οποίο δημιουργήθηκε για τα ερωτήματα 3, 9.
5. view hospitalization_bed_vw: ένα view που συνδέει πληροφορίες της κλίνης και της νοσηλείας. Δημιουργήθηκε για το ερώτημα 1.

1.2.4 Indexes και δικαιολόγηση

Για τη βελτίωση της απόδοσης της βάσης μας, δημιουργήσαμε indexes σε attributes που είναι primary keys, foreign keys και που χρησιμοποιούνται συχνά για φιλτράρισμα και joins. Η MySQL δημιουργεί αυτόματα indexes για τις δύο πρώτες περιπτώσεις, καθώς κατά τη διάρκεια της δημιουργίας του πίνακα δηλώσαμε ρητά τις περιπτώσεις που έχουμε foreign keys. Παρακάτω τονίζουμε τα indexes που προσθέσαμε χειροκίνητα, τα οποία προέκυψαν από τις παρατηρήσεις μας στα ερωτήματα, στους Triggers και στα φιλτραρίσματα των δεδομένων:

- CREATE INDEX idx_shift_date_type ON shift(shift_date,shift_type); Χρησιμοποιήθηκε με σκοπό να επιταχύνει τη λειτουργία των trigger στους πίνακες shift και shift_department_team.
- CREATE INDEX idx_procedures_date_surgeon ON medical_procedures(procedure_date, mainSurgeon_id); Χρησιμοποιήθηκε για το ερώτημα 11, όπου φιλτράρουμε βάση του procedure_date και ομαδοποιούμε με βάση το mainSurgeon_id.
- CREATE INDEX idx_med_specialty ON doctor (med_specialty); Χρησιμοποιήθηκε για το ερώτημα 2, όπου φιλτράρουμε βάση του med_specialty για να βρούμε γιατρούς της συγκεκριμένης ειδικότητας.
- shift (shift_date), faculty_shifts(faculty_id, date) Χρησιμοποιήθηκαν για τα ερωτήματα 8,12 για γρηγορότερη αναζήτηση βάση date, και θα ήταν χρήσιμοι γενικά για απάντηση ερωτημάτων διαθεσιμότητας του προσωπικού γρήγορα.
- CREATE INDEX idx_disease_type ON diagnosis(disease_type); Χρησιμοποιείται για φιλτράρισμα βάση είδους ασθένειας, όπως για παράδειγμα στο ερώτημα 14.

- CREATE INDEX idx_hosp_admission_date ON hospitalization(hospitalization_admissionDiagnosis_code); Χρησιμοποιείται για παράδειγμα στο ερώτημα 14, για join πάνω στο hospitalization_admissionDiagnosis_code.
- CREATE INDEX idx_patient_trriages ON triage(patient_id,triage_id); Γίνεται χρήση στο ερώτημα 6, για άμεση εύρεση των διαλογών που ανήκουν σε συγκεκριμένο ασθενή.
- CREATE INDEX idx_shift_dept_team ON shift_department_team(shift_id, dept_name, faculty_id); Χρήση στο ερώτημα 8, για άμεση εύρεση του μοναδικού τμήματος που ανήκει κάθε βάρδια και το κάθε μέλος προσωπικού.

2 Μέρος B, ερωτήματα SQL

Το populate της βάσης έγινε με συνθετικά δεδομένα που δημιουργήθηκαν με χρήση random data generators για τη δημιουργία ονομάτων, email, αριθμών τηλεφώνου και άλλων βασικών πεδίων (Mockaroo, Cobbl) και με κώδικα python ή τη βοήθεια LLM (Gemini, ChatGPT) για δεδομένα που τηρούν πιο σύνθετους business rules του νοσοκομείου (π.χ. για τη δημιουργία ημερομηνιών αξιολόγησης που να είναι μετά την ημερομηνία εισαγωγής στο νοσοκομείο, ιατρικών διαγνώσεων admission και discharge κάθε ασθενούς που να είναι συνεπείς απο ιατρικής άποψης και συμβαδίζουν με τους κωδικούς KEN, και άλλα).

Παραδοχές σχετικά με την ερμηνεία των ζητούμενων έχουν σημειωθεί στο αρχείο README.md.

2.1 Ερώτημα 2

Η συμπλήρωση της στήλης had_shift_this_year γίνεται με χρήση conditional logic (CASE). Στο WHERE predicate που σκανάρει βάση ημερομηνιών να γίνει χρήση του index στο shift_date, αποφύγαμε τη χρήση της συνάρτησης EXTRACT και κάναμε αντί γι'αυτό range scan για να εντοπίσουμε τις ημερομηνίες βαρδιών που βρίσκονται στο τρέχον έτος ("i= 01-01-2026").

2.2 Ερώτημα 4

2.2.1 (α) EXPLAIN-ANALYZE

```

MariaDB [hospital_v2]> EXPLAIN WITH doc_eval AS
-> (
-> SELECT p.doctor_id, AVG(e.medical_care_quality) AS avg_medical_care FROM prescription p JOIN evaluation_of_doctor e ON p.prescription_id=e.prescription_id GROUP BY p.doctor_id
-> );
-> hosp_eval AS
-> (
-> SELECT p.doctor_id, AVG(eh.overall_experience) AS avg_hosp_exp FROM prescription p JOIN evaluation_of_hospitalization eh ON p.hospitalization_id=eh.hospitalization_id GROUP BY p.doctor_id
-> );
-> SELECT d.doctor_id AS prescribing_doc_id, d.first_name, d.last_name, dr.avg_medical_care, hr.avg_hosp_exp FROM doctor_full_info d LEFT JOIN doc_eval dr ON d.doctor_id=dr.doctor_id LEFT JOIN hosp_eval hr ON d.doctor_id=hr.doctor_id WHERE d.doctor_id=42;

```

	id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	f	const	PRIMARY	PRIMARY		4	const	1	
1	PRIMARY	d	const	PRIMARY	PRIMARY		4	const	1	Using index
1	PRIMARY	<derived>2	all	NULL	NULL	NULL	NULL	NULL	139	Using where
1	PRIMARY	<derived>3	all	NULL	NULL	NULL	NULL	NULL	226	Using where; Using join buffer (flat, BNL join)
2	DERIVED	e	all	prescription_id	NULL	NULL	NULL	NULL	139	Using temporary; Using filesort
2	DERIVED	p	eq_ref	PRIMARY	PRIMARY		4	hospital_v2.e.prescription_id	1	Using temporary; Using filesort
3	DERIVED	eh	all	hospitalization_id	NULL	NULL	NULL	NULL	226	Using temporary; Using filesort
3	DERIVED	p	ref	fk_prescription_hospitalization	fk_prescription_hospitalization		4	hospital_v2.e.hospitalization_id	1	

0 rows in set (0.002 sec)

Παρατηρούμε ότι χρησιμοποιεί τα index στα primary keys και fk_prescription_hospitalization και τα rows που είναι expected να διαβαστούν κατά την εκτέλεση είναι πολλά στις περιπτώσεις που δεν χρησιμοποιείται index, ενώ μόνο 1 εαν χρησιμοποιείται index.

```

MariaDB [hospital_v2]> ANALYZE WITH doc_eval AS
-> (
-> SELECT p.doctor_id, AVG(e.medical_care_quality) AS avg_medical_care FROM prescription p JOIN evaluation_of_doctor e ON p.prescription_id=e.prescription_id GROUP BY p.doctor_id
-> ),
-> hosp_eval AS
-> (
-> SELECT p.doctor_id, AVG(eh.overall_experience) AS avg_hosp_exp FROM prescription p JOIN evaluation_of_hospitalization eh ON p.hospitalization_id=eh.hospitalization_id GROUP BY p.doctor_id
-> )
-> SELECT d.doctor_id AS prescribing_doc_id, d.first_name, d.last_name, dr.avg_medical_care, hr.avg_hosp_exp FROM doctor_full_info d LEFT JOIN doc_eval dr ON d.doctor_id=dr.doctor_id LEFT JOIN hosp_eval hr ON d.doctor_id=hr.doctor_id WHERE d.doctor_id=42;

```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	r_rows	filtered	r_filtered	Extra
1	PRIMARY	f	const	PRIMARY	PRIMARY	4	const	1	NULL	100.00	NULL	
1	PRIMARY	d	const	PRIMARY	PRIMARY	4	const	1	NULL	100.00	NULL	
1	PRIMARY	<derived>	ALL	NULL	NULL	NULL	NULL	139	44.00	100.00	1	Using index
1	PRIMARY	<derived2>	ALL	NULL	NULL	NULL	NULL	226	49.00	100.00	2.04	Using where
2	DERIVED	e	index	prescription_id	prescription_id	4	NULL	139	139.00	100.00	100.00	Using temporary; Using filesort
2	DERIVED	p	eq_ref	PRIMARY	PRIMARY	4	hospital_v2.e.prescription_id	1	1.00	100.00	100.00	
2	DERIVED	eh	ALL	hospitalization_id	hospitalization_id	4	NULL	226	226.00	100.00	100.00	Using temporary; Using filesort
3	DERIVED	p	ref	fk-prescription_hospitalization	fk-prescription_hospitalization	4	hospital_v2.eh.hospitalization_id	1	0.45	100.00	100.00	

8 rows in set (0.085 sec)

Βλέπουμε απο το analyze οτι η εκτίμηση και η πραγματική τιμή διαφέρουν σημαντικά σε κάποιες περιπτώσεις, που σημαίνει οτι ίσως απαιτείται ένας επιπλέον index σχετικά με αυτά τα attributes οστε να γίνει καλύτερη εκτίμηση, άρα και καλύτερο query plan.

2.2.2 (β) FORCE INDEX/HINT

```

MariaDB [hospital_v2]> EXPLAIN WITH doc_eval AS
-> (
-> SELECT p.doctor_id, AVG(e.medical_care_quality) AS avg_medical_care
-> FROM prescription p FORCE INDEX (idx_prescription_doctor)
-> JOIN evaluation_of_doctor e ON p.prescription_id=e.prescription_id
-> GROUP BY p.doctor_id
-> ),
-> hosp_eval AS
-> (
-> SELECT p.doctor_id, AVG(eh.overall_experience) AS avg_hosp_exp
-> FROM prescription p FORCE INDEX (idx_prescription_doctor)
-> JOIN evaluation_of_hospitalization eh ON p.hospitalization_id=eh.hospitalization_id
-> GROUP BY p.doctor_id
-> )
-> SELECT d.doctor_id AS prescribing_doc_id, d.first_name, d.last_name, dr.avg_medical_care, hr.avg_hosp_exp
-> FROM doctor_full_info d
-> LEFT JOIN doc_eval dr ON d.doctor_id=dr.doctor_id
-> LEFT JOIN hosp_eval hr ON d.doctor_id=hr.doctor_id
-> WHERE d.doctor_id=42;

```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	f	const	PRIMARY	PRIMARY	4	const	1	
1	PRIMARY	d	const	PRIMARY	PRIMARY	4	const	1	Using index
1	PRIMARY	<derived2>	ALL	NULL	NULL	NULL	NULL	400	Using where
1	PRIMARY	<derived3>	ALL	NULL	NULL	NULL	NULL	400	Using where; Using join buffer (flat, BNL join)
2	DERIVED	p	index	NULL	idx_prescription_doctor	4	NULL	400	Using index
2	DERIVED	e	eq_ref	prescription_id	prescription_id	4	hospital_v2.p.prescription_id	1	
3	DERIVED	p	ALL	NULL	NULL	NULL	NULL	1	
3	DERIVED	eh	eq_ref	hospitalization_id	hospitalization_id	4	hospital_v2.p.hospitalization_id	1	Using temporary; Using filesort

8 rows in set (0.002 sec)

Παρατηρούμε οτι το index idx_prescription_doctor ON prescription(doctor_id) που επιβάλαμε δεν βελτίωσε την εκτίμηση των rows.

```

MariaDB [hospital_v2]> EXPLAIN WITH doc_eval AS
-> (
-> SELECT p.doctor_id, AVG(e.medical_care_quality) AS avg_medical_care
-> FROM prescription p FORCE INDEX (idx_prescription_doctor)
-> JOIN evaluation_of_doctor e ON p.prescription_id=e.prescription_id
-> GROUP BY p.doctor_id
-> ),
-> hosp_eval AS
-> (
-> SELECT p.doctor_id, AVG(eh.overall_experience) AS avg_hosp_exp
-> FROM prescription p FORCE INDEX (idx_prescription_doctor)
-> JOIN evaluation_of_hospitalization eh ON p.hospitalization_id=eh.hospitalization_id
-> GROUP BY p.doctor_id
-> )
-> SELECT d.doctor_id AS prescribing_doc_id, d.first_name, d.last_name, dr.avg_medical_care, hr.avg_hosp_exp
-> FROM doctor_full_info d
-> LEFT JOIN doc_eval dr ON d.doctor_id=dr.doctor_id
-> LEFT JOIN hosp_eval hr ON d.doctor_id=hr.doctor_id
-> WHERE d.doctor_id=42;

```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	PRIMARY	f	const	PRIMARY	PRIMARY	4	const	1	
1	PRIMARY	d	const	PRIMARY	PRIMARY	4	const	1	Using index
1	PRIMARY	<derived2>	ALL	NULL	NULL	NULL	NULL	400	Using where
1	PRIMARY	<derived3>	ALL	NULL	NULL	NULL	NULL	400	Using where; Using join buffer (flat, BNL join)
2	DERIVED	p	index	NULL	idx_prescription_doctor	4	NULL	400	Using index
2	DERIVED	e	eq_ref	prescription_id	prescription_id	4	hospital_v2.p.prescription_id	1	
3	DERIVED	p	ALL	NULL	NULL	NULL	NULL	400	Using temporary; Using filesort
3	DERIVED	eh	eq_ref	hospitalization_id	hospitalization_id	4	hospital_v2.p.hospitalization_id	1	

8 rows in set (0.002 sec)

Επιπλέον οτι και η πραγματική τιμή r_rows είναι μεγαλύτερη σε αυτή την περίπτωση. Επομένως αυτό το index δεν είναι optimal στην περίπτωση αυτή.

2.2.3 (γ) Σύγκριση εκτιμώμενου και πραγματικού χρόνου εκτέλεσης μεταξύ των δύο εκδοχών

Και στις δύο περιπτώσεις η εκτίμηση και ο πραγματικός χρόνος διαφέρουν σημαντικά και αυτό πιθανώς να οφείλεται στο οτι ο index είναι suboptimal αλλά και στον μικρό όγκο των δεδομένων της βάσης.

2.2.4 (δ) Συμπεράσματα

Συμπεραίνουμε ότι η χρήση index δεν κάνει απαραίτητα διαφορά στην εκτίμηση ούτε και στην πραγματική τιμή. Επομένως ο index που ελέγξαμε έγινε drop γιατί δεν βελτίωνε την απόδοση και αν παρέμενε πιθανώς να καθυστερούσε τη βάση σε εισαγωγές/updates.

2.3 Ερώτημα 6

Η σειρά των join επιλέγεται ως: πρώτα triage join hospitalization για να μειωθεί γρήγορα η διάσταση του μεγαλύτερου πίνακα που είναι το triage, μετά join evaluation_of_hospitalization και join cost.

2.3.1 (α) EXPLAIN-ANALYZE

```
MariaDB [hospital_v2]> EXPLAIN SELECT h.hospitalization_id,
-> h.admission_date AS admission_history,
-> h.hospitalization_admissionDiagnosis_code AS admission_diagnosis,
-> h.hospitalization_dischargeDiagnosis_code AS discharge_diagnosis,
-> (c.KEN_cost+c.extra_cost*ALOS) AS total_hosp_cost,
-> ((e.nursing_care_quality+e.cleanliness+e.food+e.overall_experience)/4.0) AS average_of_hosp_evaluation
-> FROM triage t
-> JOIN hospitalization h ON t.triage_id=h.triage_id
-> JOIN evaluation_of_hospitalization e ON h.hospitalization_id=e.hospitalization_id
-> JOIN cost c ON h.cost_id=c.cost_id
-> WHERE t.patient_id=125;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	t	ref	PRIMARY,idx_patient_trriages	idx_patient_trriages	4	const	7	Using index
1	SIMPLE	h	eq_ref	PRIMARY,triage_id,fk_hospitalization_cost	triage_id	4	hospital_v2.t.triage_id	1	
1	SIMPLE	e	eq_ref	hospitalization_id	hospitalization_id	4	hospital_v2.h.hospitalization_id	1	
1	SIMPLE	c	eq_ref	PRIMARY	PRIMARY	4	hospital_v2.h.cost_id	1	

4 rows in set (0.036 sec)

Παρατηρούμε ότι χρησιμοποιεί τα indexes και οι εκτιμήσεις των rows είναι μικρές.

```
ANALYZE SELECT h.hospitalization_id,
-> h.admission_date AS admission_history,
-> h.hospitalization_admissionDiagnosis_code AS admission_diagnosis,
-> h.hospitalization_dischargeDiagnosis_code AS discharge_diagnosis,
-> (c.KEN_cost+c.extra_cost*ALOS) AS total_hosp_cost,
-> ((e.nursing_care_quality+e.cleanliness+e.food+e.overall_experience)/4.0) AS average_of_hosp_evaluation
-> FROM triage t
-> JOIN hospitalization h ON t.triage_id=h.triage_id
-> JOIN evaluation_of_hospitalization e ON h.hospitalization_id=e.hospitalization_id
-> JOIN cost c ON h.cost_id=c.cost_id
-> WHERE t.patient_id=125;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	r_rows	filtered	r_filtered	Extra
1	SIMPLE	t	ref	PRIMARY,idx_patient_trriages	idx_patient_trriages	4	const	7	7.00	100.00	100.00	Using index
1	SIMPLE	h	eq_ref	PRIMARY,triage_id,fk_hospitalization_cost	triage_id	4	hospital_v2.t.triage_id	1	1.00	100.00	100.00	
1	SIMPLE	e	eq_ref	hospitalization_id	hospitalization_id	4	hospital_v2.h.hospitalization_id	1	0.71	100.00	100.00	
1	SIMPLE	c	eq_ref	PRIMARY	PRIMARY	4	hospital_v2.h.cost_id	1	1.00	100.00	100.00	

4 rows in set (0.039 sec)

Παρατηρούμε καλές εκτιμήσεις.

2.3.2 (β) FORCE INDEX/HINT

```
MariaDB [hospital_v2]> EXPLAIN SELECT h.hospitalization_id,
-> h.admission_date AS admission_history,
-> h.hospitalization_admissionDiagnosis_code AS admission_diagnosis,
-> h.hospitalization_dischargeDiagnosis_code AS discharge_diagnosis,
-> (c.KEN_cost+c.extra_cost*ALOS) AS total_hosp_cost,
-> ((e.nursing_care_quality+e.cleanliness+e.food+e.overall_experience)/4.0) AS average_of_hosp_evaluation
-> FROM triage t FORCE INDEX (PRIMARY)
-> JOIN hospitalization h ON t.triage_id=h.triage_id
-> JOIN evaluation_of_hospitalization e ON h.hospitalization_id=e.hospitalization_id
-> JOIN cost c ON h.cost_id=c.cost_id
-> WHERE t.patient_id=125;
```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	Extra
1	SIMPLE	e	ALL	hospitalization_id	NULL	NULL	NULL	226	
1	SIMPLE	h	eq_ref	PRIMARY,triage_id,fk_hospitalization_cost	PRIMARY	4	hospital_v2.e.hospitalization_id	1	
1	SIMPLE	c	eq_ref	PRIMARY	PRIMARY	4	hospital_v2.h.cost_id	1	
1	SIMPLE	t	eq_ref	PRIMARY	PRIMARY	4	hospital_v2.h.triage_id	1	Using where

4 rows in set (0.001 sec)

Παρατηρούμε ότι το index idx_prescription_doctor ON prescription(doctor_id) που επιβάλαμε δεν βελτίωσε την απόδοση (διάβασε πιο πολλές γραμμές στην πραγματικότητα).

```

MariaDB [hospital_v2]> ANALYZE SELECT h.hospitalization_id,
-> h.admission_date AS admission_history,
-> h.hospitalization_admissionDiagnosis_code AS admission_diagnosis,
-> h.hospitalization_dischargeDiagnosis_code AS discharge_diagnosis,
-> (c.KEN_cost+c.extra_cost_AL05) AS total_hosp_cost,
-> ((e.nursing_care_quality+e.cleanliness+e.food+e.overall_experience)/4.0) AS average_of_hosp_evaluation
-> FROM triage t FORCE INDEX (PRIMARY)
-> JOIN hospitalization h ON t.triage_id=h.triage_id
-> JOIN evaluation_of_hospitalization e ON h.hospitalization_id=e.hospitalization_id
-> JOIN cost c ON h.cost_id=c.cost_id
-> WHERE t.patient_id=125;

```

id	select_type	table	type	possible_keys	key	key_len	ref	rows	r_rows	filtered	r_filtered	Extra
1	SIMPLE	e	ALL	hospitalization_id	NULL	NULL	NULL	226	226.00	100.00	100.00	
1	SIMPLE	h	eq_ref	PRIMARY, triage_id, fk_hospitalization_cost	PRIMARY	4	hospital_v2.e.hospitalization_id	1	1.00	100.00	100.00	
1	SIMPLE	c	eq_ref	PRIMARY	PRIMARY	4	hospital_v2.h.cost_id	1	1.00	100.00	100.00	
1	SIMPLE	t	eq_ref	PRIMARY	PRIMARY	4	hospital_v2.h.triage_id	1	1.00	100.00	2.21	Using where

4 rows in set (0.038 sec)

2.3.3 (γ) Σύγκριση εκτιμώμενου και πραγματικού χρόνου εκτέλεσης μεταξύ των δύο εκδοχών

Και στις δύο περιπτώσεις εδώ είναι κοντά με μεγαλύτερη ακρίβεια στη περίπτωση force index

2.3.4 (δ) Συμπεράσματα

Συμπεραίνουμε ότι ακόμη κι αν ο optimizer αρχικά κάνει χειρότερη εκτίμηση (πιο μακριά από την πραγματική), τελικά μπορεί να έχει επιλέξει πιο αποδοτικό execution plan, αφού διαβάζει λιγότερες γραμμές.

2.4 Ερώτημα 8

Το φιλτράρισμα των χρήσιμων tuples γίνεται νωρίτερα απαλορίζοντας όλες τις μη επιθυμητές ημερομηνίες ήδη από το join predicate.

2.5 Ερώτημα 11

Το φιλτράρισμα θα μπορούσε να είχε γίνει με YEAR(procedure_date)=2026, αλλά αντικαταστάθηκε με BETWEEN '2026-01-01' AND '2026-12-31' για να είναι πιο αποδοτικό.

2.6 Ερώτημα 12

Χρησιμοποιούμε union all για να δημιουργήσουμε ένα πίνακα με όλο το προσωπικό, το είδος (class: doctor/nurse/administartive faculty) και τον ρόλο τους (subclass: εξαρτάται από το class). Επίσης όταν γίνεται το join του faculty_subclasses με το faculty_shifts, φιλτράρουμε ώστε να βρούμε την πληροφορία μόνο για την επιθυμητή εβδομάδα, και να μην δημιουργηθεί πίνακας με πληροφορίας για όλα τα dates (δηλαδή προτιμήθηκε η δημιουργία αυτού του μικρότερου ισοδύναμου πίνακα, αντί για έναν μεγαλύτερο με πληροφορία για όλες τις ημερομηνίες, τον οποίο θα πρέπει να φιλτράρουμε με where).